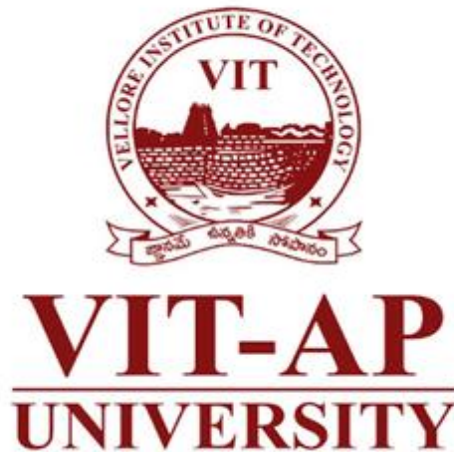




Data Visualization Lab Report

Submitted by

M Vamsidhar – 23MSD7001

Under the guidance of

DR. PROF. AVADHESH KUMAR

JUNE, 2024

CONTENTS

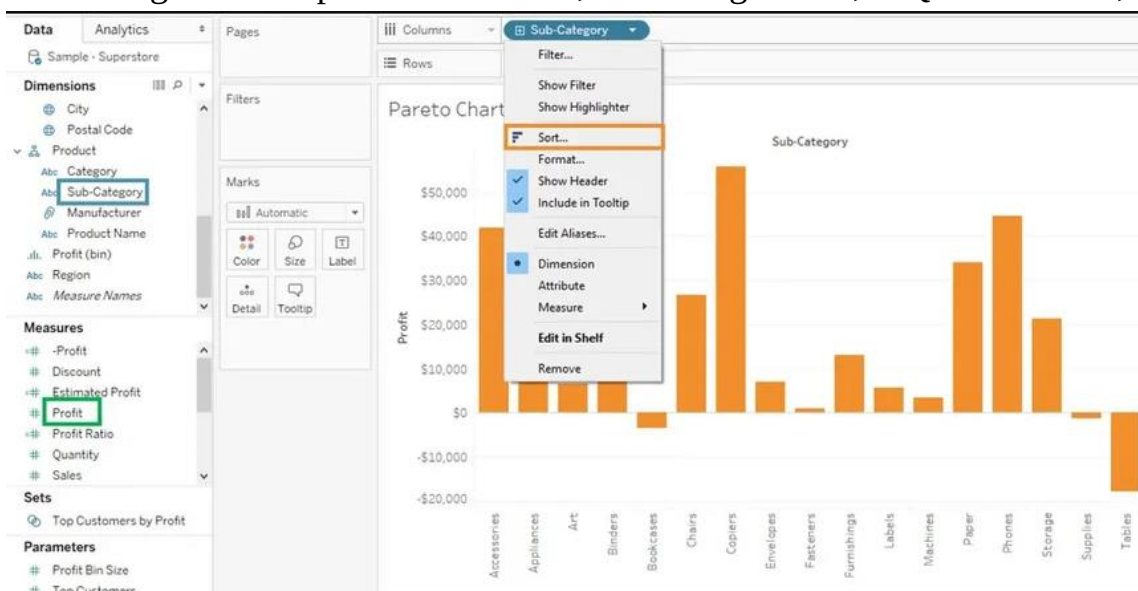
SL NO	EXPERIMENT
1	Introduction to Tableau
2	HR Analytics dashboard using tableau
3	P2 churn modelling using tableau
4	Fitting of straight and curve line
5	Plotting Stacked bar chart
6	Visualisation using ggplot
7	Plotting Dendrogram for Hierarchical Agglomerative Clustering
8	Geographical Plotting
9	Multidimensional Scaling

Introduction to Tableau



The goal of Tableau's founding in 2003 was to simplify the visualization and comprehension of complex data for users. It is an extremely potent data visualization tool that has completely changed how people and organizations evaluate and display their data. With a variety of data sources, it enables users to create visually appealing and interactive dashboards and reports. Tableau's versatility, ease of use, and capacity to manage massive amounts of data have contributed to its recent surge in popularity.

Connecting to multiple data sources, including Excel, SQL databases, and



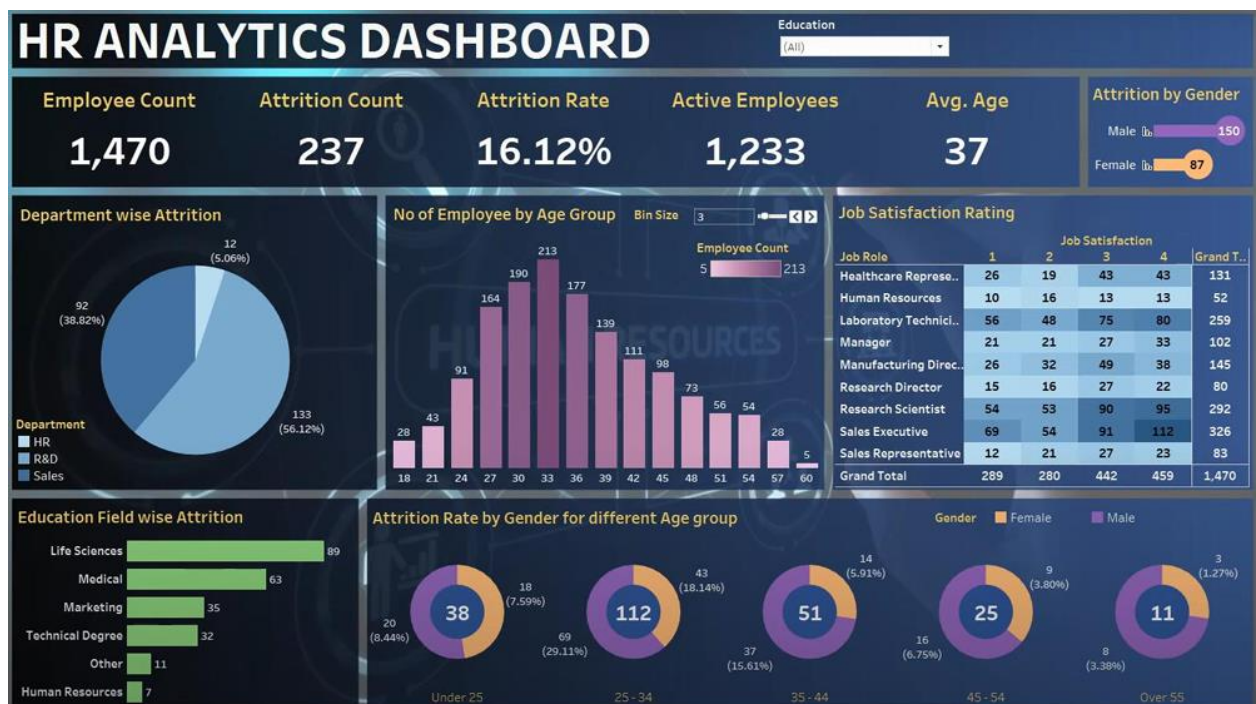
cloud-based applications like Google Analytics and Salesforce, is one of Tableau's primary features.

HR Analytics Dashboard Using Tableau

ABOUT THE DATASET

Data about employees leaving an organization is referred to as attrition data, sometimes called employee turnover data. It offers information about the frequency and causes of employee attrition. The dataset is used to examine and comprehend trends, patterns, and variables that affect an organization's employee turnover rate. This includes a number of characteristics that allow for the prediction of attrition rate.

Dashboard is given below:



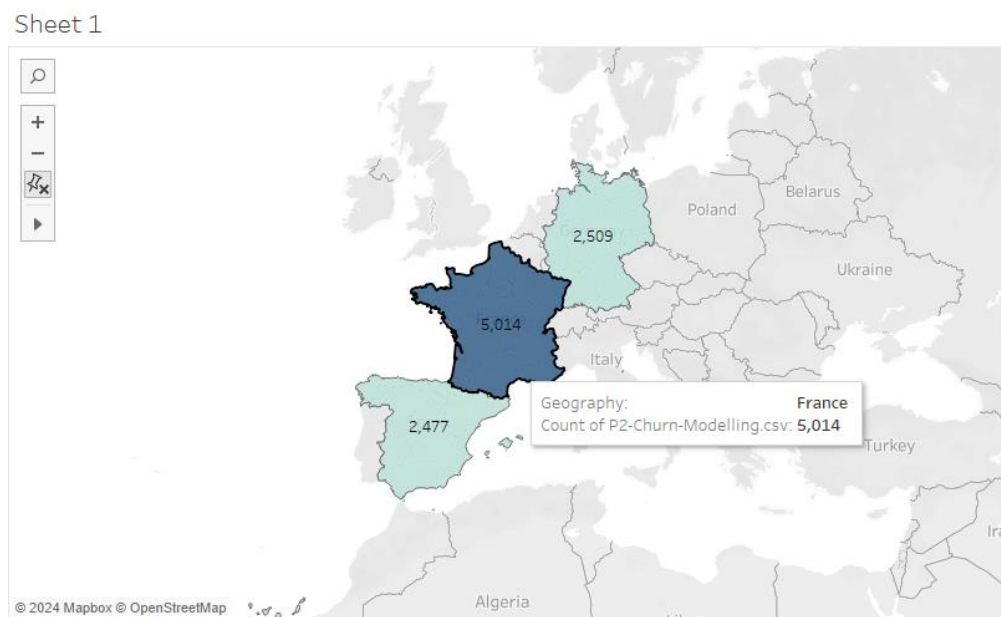
Insights

1. The company has total 1470 employees out of which attrition count is 237.
2. Attrition rate is 16.12% with 1233 active employees.
3. If we visualise the pie chart in the dashboard for department wise attrition, R&D has highest attrition rate of 56.12% followed by Sales department 38.82% attrition rate and HR department has lowest attrition rate of 5.06%.
4. And if we compare attrition count by education field then life sciences has highest attrition count and human resources has lowest attrition count.
5. Also, attrition count of male is more than that of female.

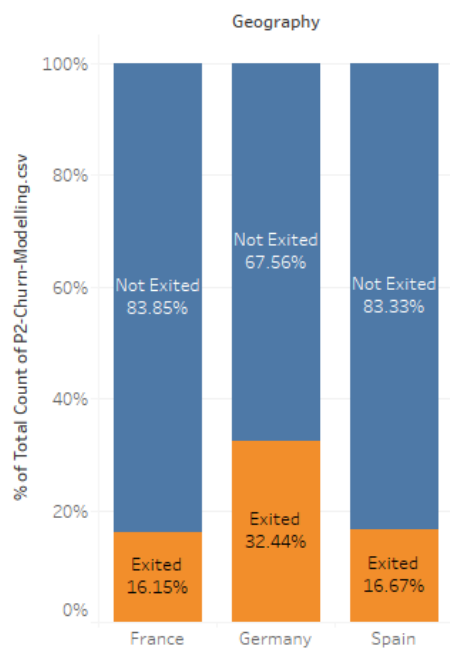
P2 Churn Modelling Using Tableau

The rate at which clients discontinue doing business with a company is called the churn rate, sometimes referred to as the rate of attrition or customer churn. The most popular way to express it is as the proportion of service users who cancel within a specified time frame.

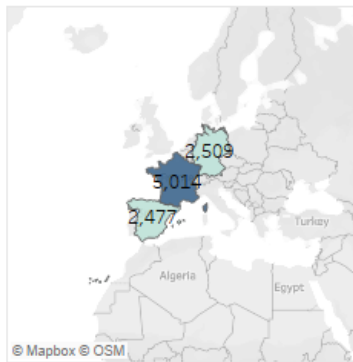
Here we performed churn prediction on the dataset using Tableau.



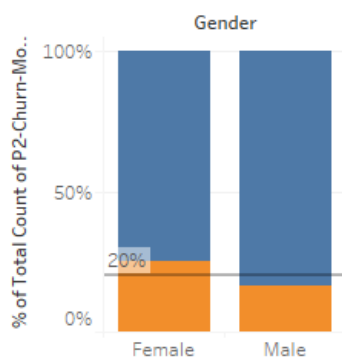
Sheet 4



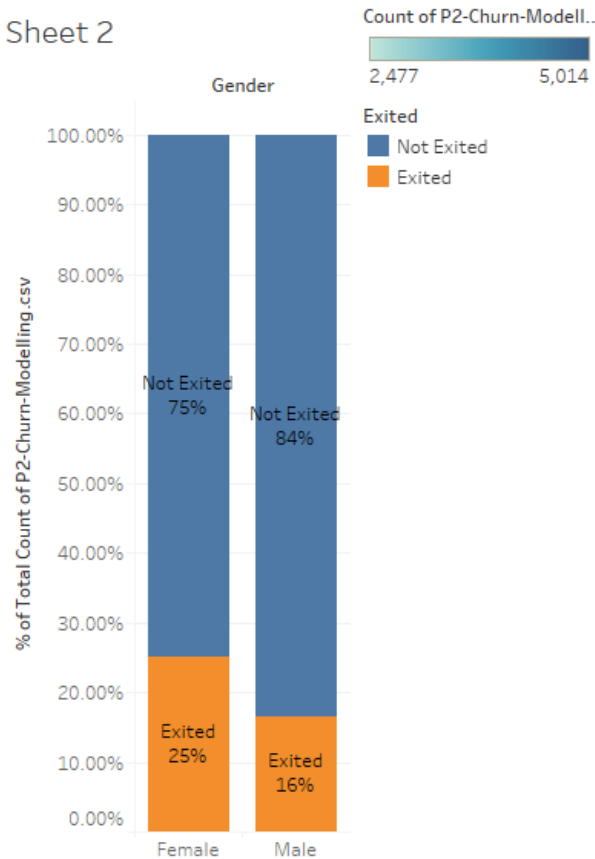
Sheet 1



Gender Analysis



Sheet 2



By exploring the data and carrying out in depth analysis and visualization, the data has provided us with great insights into why the company is having a high churn rate.

Insights

1. We imported the dataset and we visualise all the columns of the dataset using bar chart, pie chart , stacked bar chart etc.
2. 79 % female has credit card and they continued purchasing the products and 21% does not have credit card and they stop purchasing the products.
3. Similarly, 80% male had credit card and they continue to purchase the products and 20% doesn't have credit card and they stopped purchasing the products.
4. From geographical analysis, we can observe that Germany has highest churn rate of 32% as compare to Spain and France.
5. By analysing age distribution of customers, it is observed that customers in the age group 30-35 have highest churn count as compare to other age groups.

Plotting Exponential Curve And Straight Line

Using python library numpy and matplotlib, we plotted exponential , straight line , sine and cosine curve.

The code is given as follows:

```
import matplotlib.pyplot as plt
import numpy as np

# Define the x-axis values
x = np.linspace(0, 5, 100) # 100 points between 0 and 5

# Define the exponential function
y_expo = 2**x # Base 2 exponential function

# Define the straight line function (y = mx + b)
m = 1 # Slope
b = 3 # y-intercept
y_line = m * x + b

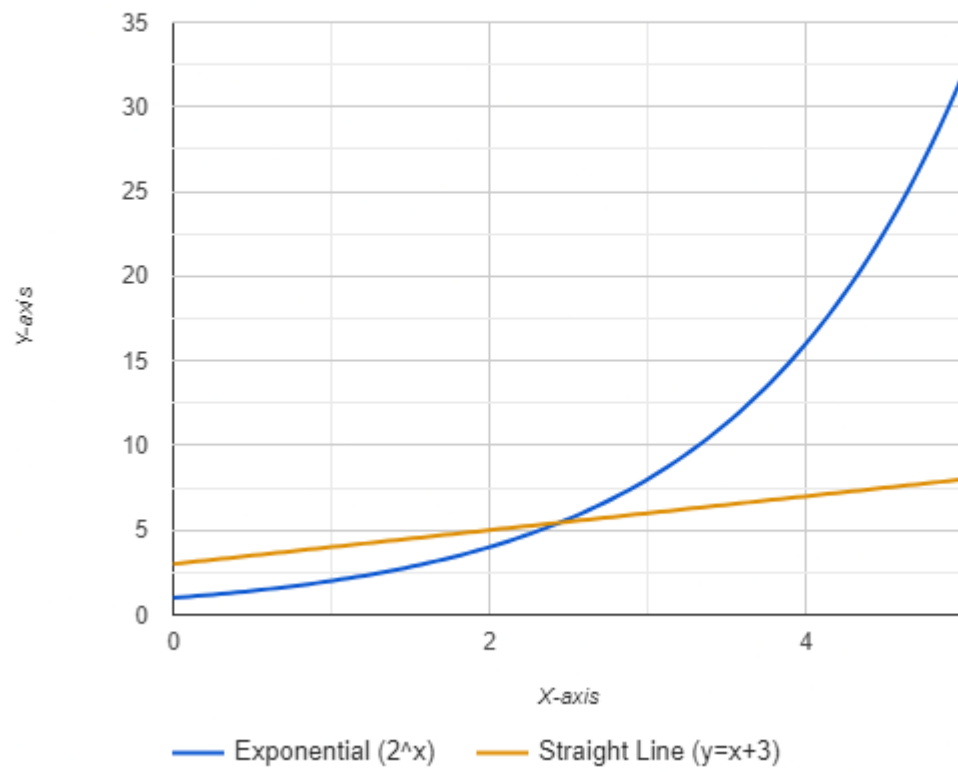
# Plot the functions
plt.plot(x, y_expo, label='Exponential (2^x)')
plt.plot(x, y_line, label='Straight Line (y=x+3)')

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Exponential vs. Straight Line')

# Add legend
plt.legend()

# Show the plot
plt.grid(True)
plt.show()
```


Exponential vs. Straight Line



Plotting Stacked Bar Chart

The code is given below:

```
import matplotlib.pyplot as plt
import numpy as np

# Define the x-axis values
x = np.linspace(0, 5, 100) # 100 points between 0 and 5

# Define the exponential function
y_expo = 2**x # Base 2 exponential function

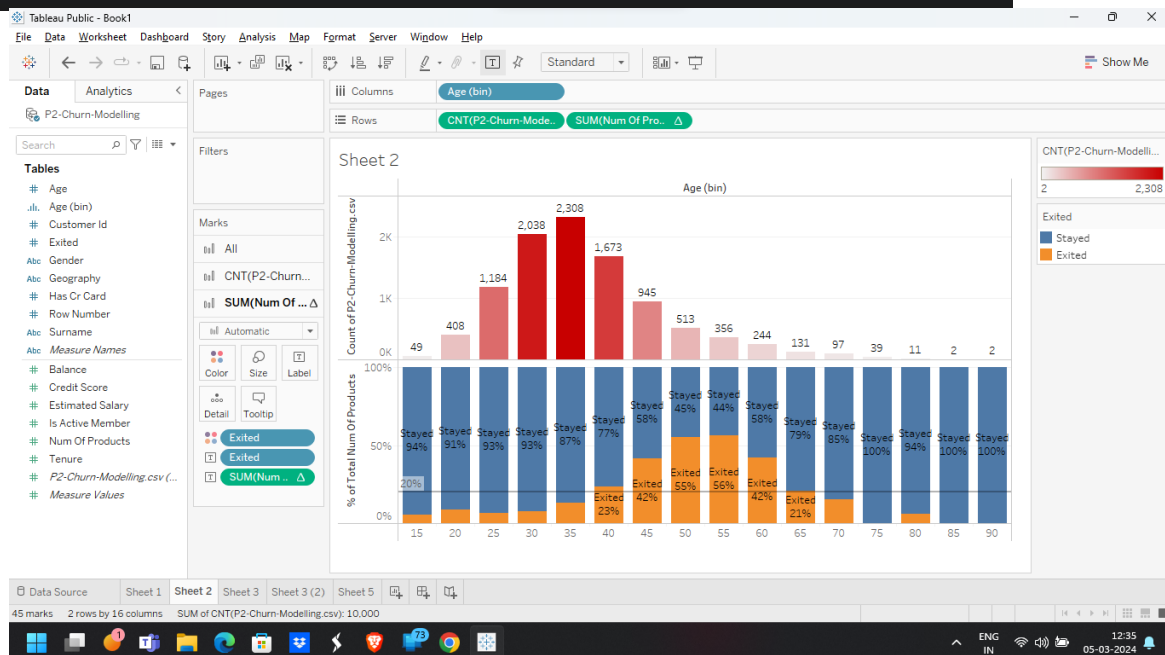
# Define the straight line function (y = mx + b)
m = 1 # Slope
b = 3 # y-intercept
y_line = m * x + b

# Plot the functions
plt.plot(x, y_expo, label='Exponential (2^x)')
plt.plot(x, y_line, label='Straight Line (y=x+3)')

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Exponential vs. Straight Line')

# Add legend
plt.legend()

# Show the plot
plt.grid(True)
plt.show()
```

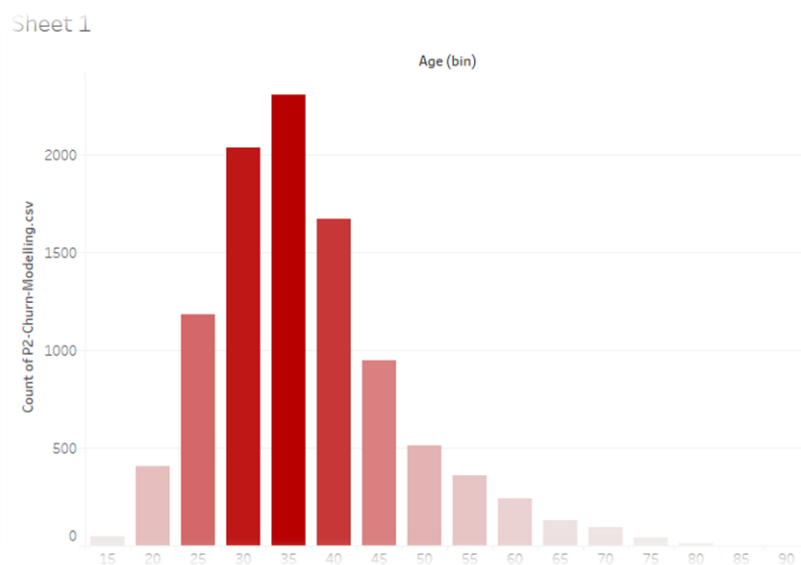


Insights

1. Imported matplotlib library of python for bar graph plotting.
2. Using bar graph, GDP of 5 countries Qatar, Luxembourg, Brunei, UAE and Switzerland has been plotted.
3. From the bar graph it is observed that Luxembourg has highest GDP above 10\$ billion and Brunei has lowest GDP in the range of 2 to 4\$ billion.

Insights

1. Plotted stacked bar chart of GDP contribution by 3 different sectors manufacturing, service, agriculture of above mentioned 5 countries.



2. It is observed that service sector in Switzerland has highest GDP followed by Luxembourg, UAE, Qatar and Brunei.
3. Manufacturing sector in Switzerland has lowest GDP among all 5 countries in this sector and Qatar and Luxembourg has same GDP contribution in manufacturing sector of approx. 3.5 \$ billion.
4. Qatar and Brunei have almost same GDP contribution in agriculture sector.

Visualisation using ggplot

Using plotnine, create a plot to show the evolution of the population through the years.

The code is given below:

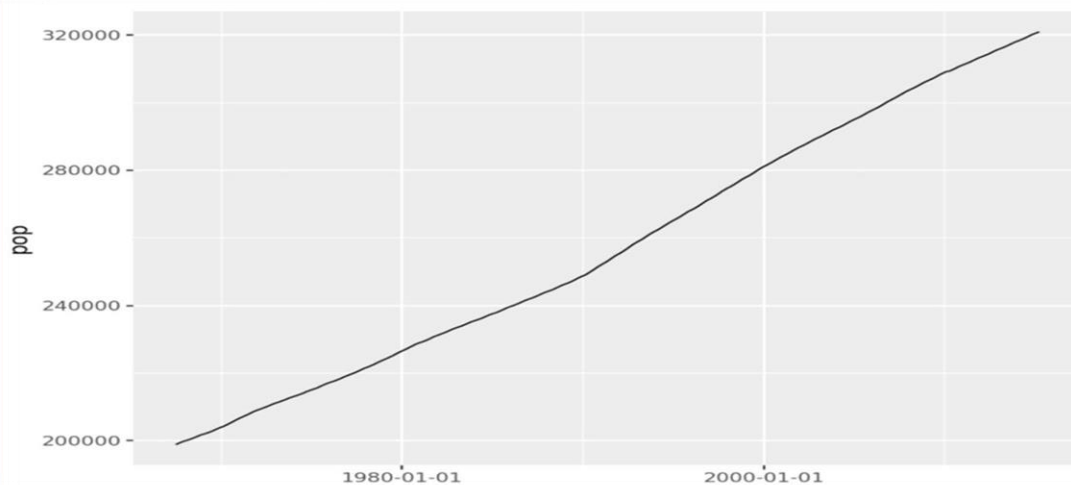
```
1 !pip install plotnine -q
2 import warnings
3 warnings.filterwarnings('ignore')

1 from plotnine.data import economics

1 economics
```

	date	pce	pop	psavert	uempmed	unemploy
0	1967-07-01	507.4	198712	12.5	4.5	2944
1	1967-08-01	510.5	198911	12.5	4.7	2945
2	1967-09-01	516.3	199113	11.7	4.6	2958
3	1967-10-01	512.9	199311	12.5	4.9	3143
4	1967-11-01	518.1	199498	12.5	4.7	3066
...	...	...	...	...	...	...
569	2014-12-01	12122.0	320201	5.0	12.6	8688
570	2015-01-01	12080.8	320367	5.5	13.4	8979
571	2015-02-01	12095.9	320534	5.7	13.1	8705
572	2015-03-01	12161.5	320707	5.2	12.2	8575
573	2015-04-01	12158.9	320887	5.6	11.7	8549

```
1 from plotnine import ggplot, aes, geom_line
2 (
3     ggplot(economics) # What data to use
4     + aes(x="date", y="pop") # What variable to use
5     + geom_line() # Geometric object to use for drawing
6 )
```



Plotting Dendrogram for Hierarchical Agglomerative Clustering

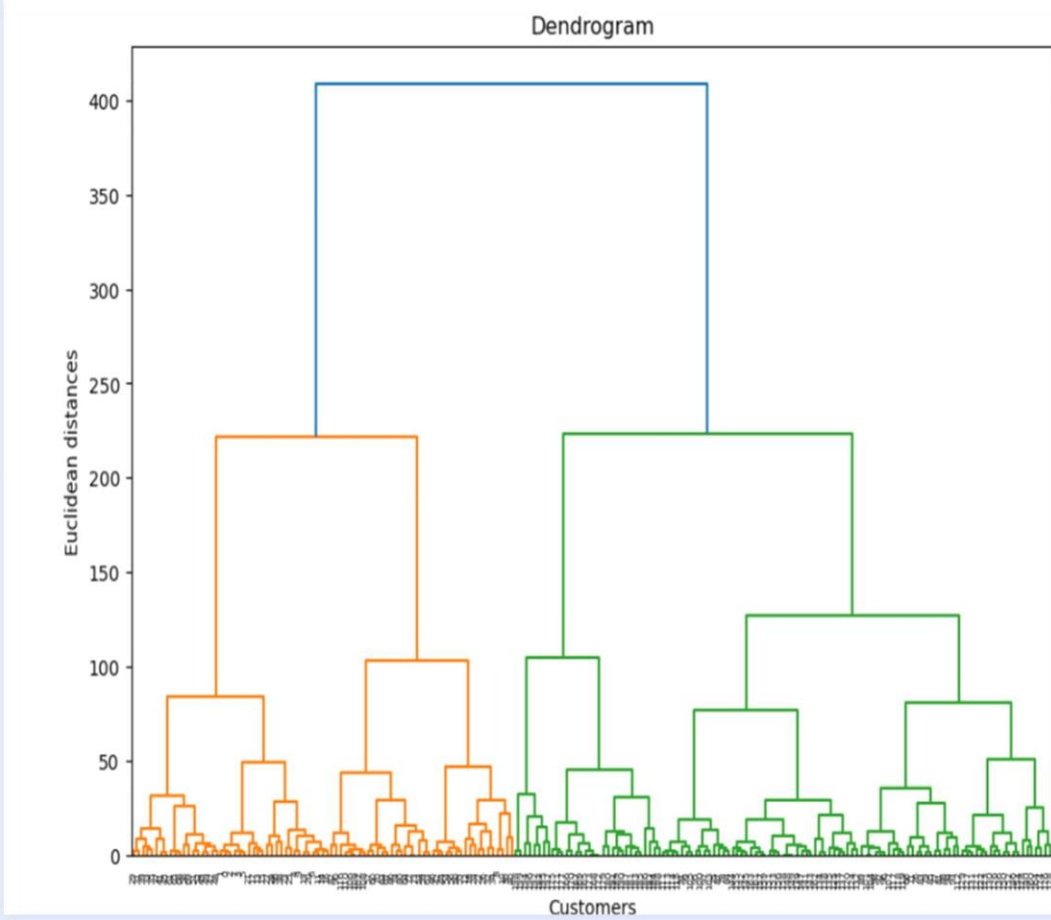
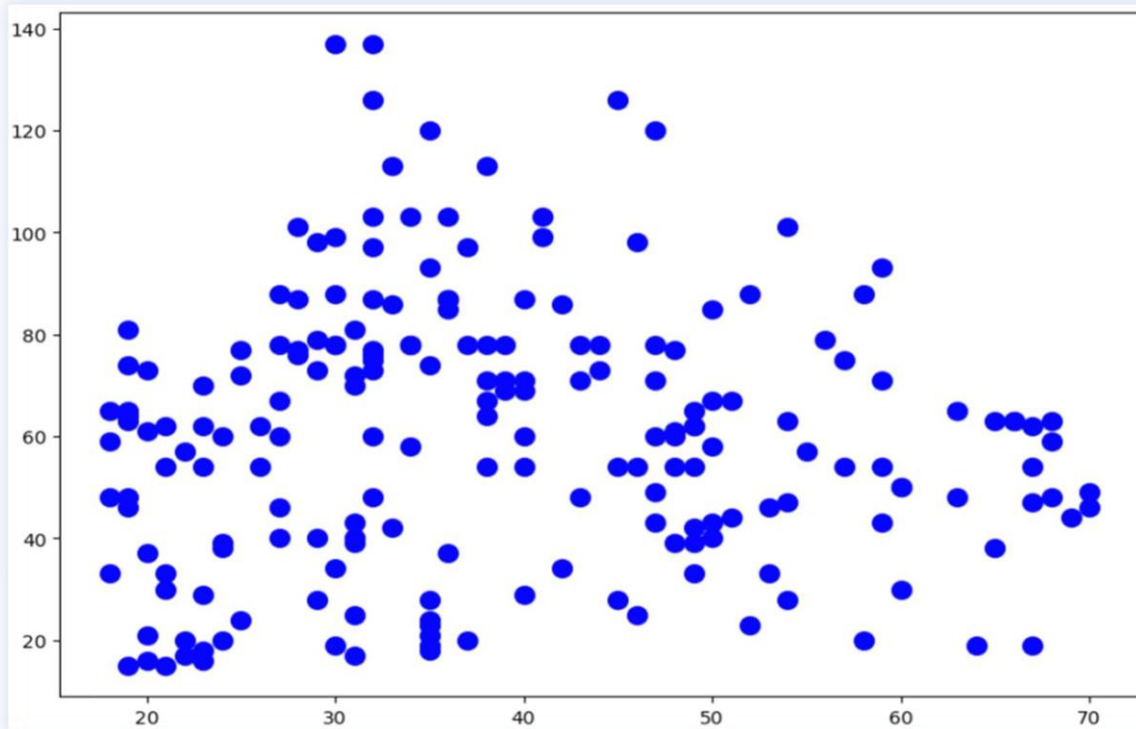
The agglomerative clustering is the most common type of hierarchical clustering used to group objects in clusters based on their similarity. Here we used the dataset of mall customers and plotted the dendrogram to get optimal no of clusters.

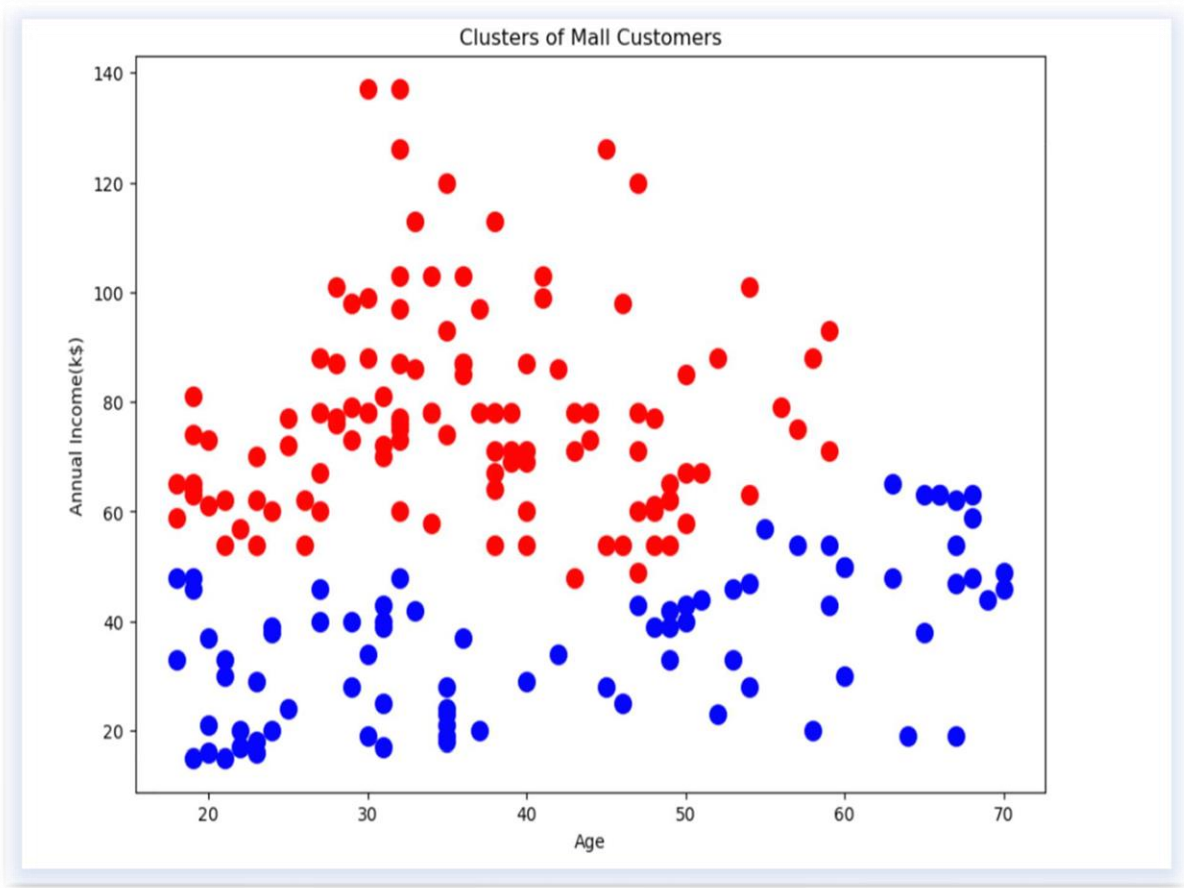
```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 %matplotlib inline
```

```
1 df = pd.read_csv("D:/datasets/Mall_Customers.csv")
2 df.head()
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
1 datasubset = df.loc[:, ["Age", "Annual Income (k$)"]]
2 plt.figure(figsize=(10, 7))
3 plt.scatter(datasubset[["Age"]], datasubset[["Annual Income (k$)"]], s=100, c='blue')
4
5 import scipy.cluster.hierarchy as sch
6 plt.figure(figsize=(10, 7))
7
8
9 dendrogram = sch.dendrogram(sch.linkage(datasubset, method = "ward"))
10 plt.title('Dendrogram')
11 plt.xlabel('Customers')
12 plt.ylabel('Euclidean distances')
13 plt.show()
14
15 from sklearn.cluster import AgglomerativeClustering
16 cluster = AgglomerativeClustering(n_clusters=2, affinity='euclidean', linkage='ward')
17 cluster.fit_predict(datasubset)
18 cl = cluster.fit_predict(datasubset)
19
20 from sklearn.metrics import silhouette_score
21
22 silhouette_score(datasubset, cl)
23
24 X = datasubset.values
25 plt.figure(figsize=(10, 7))
26 plt.scatter(X[cl==0, 0], X[cl==0, 1], s=100, c='red', label = 'Cluster 1')
27 plt.scatter(X[cl==1, 0], X[cl==1, 1], s=100, c='blue', label = 'Cluster 2')
28 plt.title('Clusters of Mall Customers')
29 plt.xlabel('Age')
30 plt.ylabel('Annual Income(k$)')
31 plt.show()
```





Geographical Plotting

Geographical plotting is a method of displaying data on a global scale as well as for states of a country, often in a colorful manner. It plays a significant role in data analysis and visualization.

Visualisation of geographical plot using agriculture dataset:

We used plotly which is a famous library used for creating interactive plotting and dashboards in Python.

```
!pip install chart_studio
import chart_studio.plotly as py
import plotly.offline as po
import plotly.graph_objs as pg
```

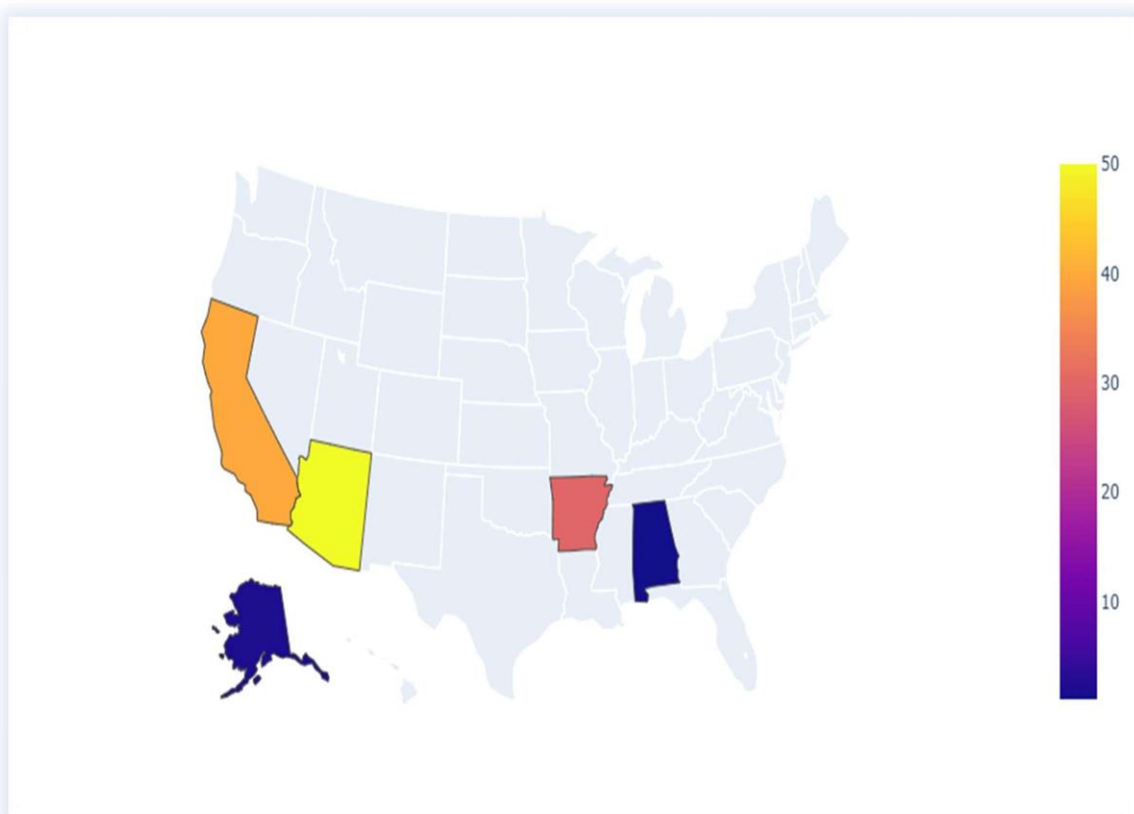
```
1 data = pd.read_csv("C:/Users/91981/Downloads/agri.csv")
2 data.head()
```

code	state	category	total exports	beef	pork	poultry	dairy	fruits fresh	fruits proc	total fruits	veggies fresh	veggies proc	total veggies	corn	wheat	cotton	text
AL	Alabama	state	1390.63	34.4	10.6	481.0	4.06	8.0	17.1	25.11	5.5	8.9	14.33	34.9	70.0	317.61	Alabama Beef 34.4 Dairy 4.06 Fruits 25.1...
AK	Alaska	state	13.31	0.2	0.1	0.0	0.19	0.0	0.0	0.00	0.6	1.0	1.56	0.0	0.0	0.00	Alaska Beef 0.2 Dairy 0.19 Fruits 0.0 Ve...
AZ	Arizona	state	1463.17	71.3	17.9	0.0	105.48	19.3	41.0	60.27	147.5	239.4	386.91	7.3	48.7	423.95	Arizona Beef 71.3 Dairy 105.48 Fruits 60...
AR	Arkansas	state	3586.02	53.2	29.4	562.9	3.53	2.2	4.7	6.88	4.4	7.1	11.45	69.5	114.5	665.44	Arkansas Beef 53.2 Dairy 3.53 Fruits 6.8...
CA	California	state	16472.88	228.7	11.1	225.4	929.95	2791.8	5944.6	8736.40	803.2	1303.5	2106.79	34.6	249.3	1064.95	California Beef 228.7 Dairy 929.95 Fruit...


```

1 po.init_notebook_mode(connected = True)
2 data = dict(type = 'choropleth',
3             locations = ['AL', 'AK', 'AR', 'CA', 'AZ'],
4             locationmode = 'USA-states',
5             z = [1, 2, 30, 40, 50],
6             text = ['alabama', 'alaska', 'arizona', 'pugger', 'california'])
7
8 layout = dict(geo = {'scope': 'usa'})
9
10 x = pg.Figure(data = [data] ,
11                layout = layout)
12 po.iplot(x)

```



Multidimensional Scaling

Multidimensional scaling (MDS) is a dimensionality reduction technique that is used to project high-dimensional data onto a lower-dimensional space while preserving the pairwise distances between the data points as much as possible. MDS is based on the concept of distance and aims to find a projection of the data that minimizes the differences between the distances in the original space and the distances in the lower-dimensional space.

```
1 colors = ['r', 'g', 'b', 'c', 'm']
2 size = [64, 64, 64, 64, 64]
3 fig = plt.figure(2, (10,5))
4 ax = fig.add_subplot(111, projection='3d')
5 plt.scatter(X[:,0], X[:,1], zs=X[:,2], s=size, c=colors)
6
7
8 plt.scatter(X_transform[:,0], X_transform[:,1])
9
10
11
12
13 dist_manhattan = manhattan_distances(X)
14 mds = MDS(dissimilarity='precomputed', random_state=0)
15 # Get the embeddings
16 X_transform_L1 = mds.fit_transform(dist_manhattan)
17
18 X_transform_L1
19
20 stress = mds.stress_
21 print(stress)
22
23 colors = ['r', 'g', 'b', 'c', 'm']
24 size = [64, 64, 64, 64, 64]
25 fig = plt.figure(2, (10,5))
26 ax = fig.add_subplot(221, projection='3d')
27 plt.scatter(X[:,0], X[:,1], zs=X[:,2], s=size, c=colors)
28 plt.title('Original Points')
29
30 ax = fig.add_subplot(222)
31 plt.scatter(X_transform[:,0], X_transform[:,1], s=size, c=colors)
32 plt.title('Embedding in 2D')
33 fig.subplots_adjust(left=.25, right = 2.5, wspace=.1, hspace=0.1)
34 plt.show()
35
36 ax = fig.add_subplot(223)
37 plt.scatter(X_transform_L1[:,0], X_transform_L1[:,1], s=size, c=colors)
38 plt.title('Embedding in 2D')
39 plt.show()
```

```
[[ 0.72521687  0.52943352]
 [ 0.61640884 -0.48411805]
 [-0.9113603  -0.47905115]
 [-0.2190564   0.71505714]
 [-0.21120901 -0.28132146]]
0.18216844548575467
```

```

1 from sklearn.manifold import MDS
2 from matplotlib import pyplot as plt
3 import sklearn.datasets as dt
4 #import seaborn as sns
5 import numpy as np
6 from sklearn.metrics.pairwise import manhattan_distances, euclidean_distances
7 #from matplotlib.offsetbox import OffsetImage, AnnotationBbox
8 MDS(n_components=3, metric=True, n_init=4, max_iter=300, verbose=0, eps=0.001, n_jobs=-1, random_state=0, dissimilarity='p
9
10 X = np.array([[0, 0, 0], [0, 0, 1], [1, 1, 1], [0, 1, 0], [0, 1, 1]])
11 mds = MDS(random_state=0)
12 X_transform = mds.fit_transform(X)
13 print(X_transform)
14 stress = mds.stress_
15 print(stress)
16
17

```

